Farma DB systém

2025 **Anna Berezan**

Obsah

Úvod 3

Druhá časť zadania 4

1.1. 2 pohľady s netriviálnym selektom nad jednou tabuľkou (nestačí použiť iba vymenovanie stĺpcov, treba použiť "niečo navyše" napr.: vstavané funkcie); 4

1.2. 3 pohľady so spájaním tabuliek (1x spojenie 3 tabuliek, 1x outer join, 1x spojenie aspoň 2 tabuliek); 6

1.3. 2 pohľady s použitím agregačných funkcií alebo zoskupenia; 11

1.4. 1 pohľad s použitím množinových operácií; 14

1.5. 2 pohľady s použitím netriviálnych vnorených selektov; 15

Vytvorte 1 sekvenciu na generovanie primárnych kľúčov a trigger, ktorý bude vkladať hodnoty do príslušných tabuliek; 17

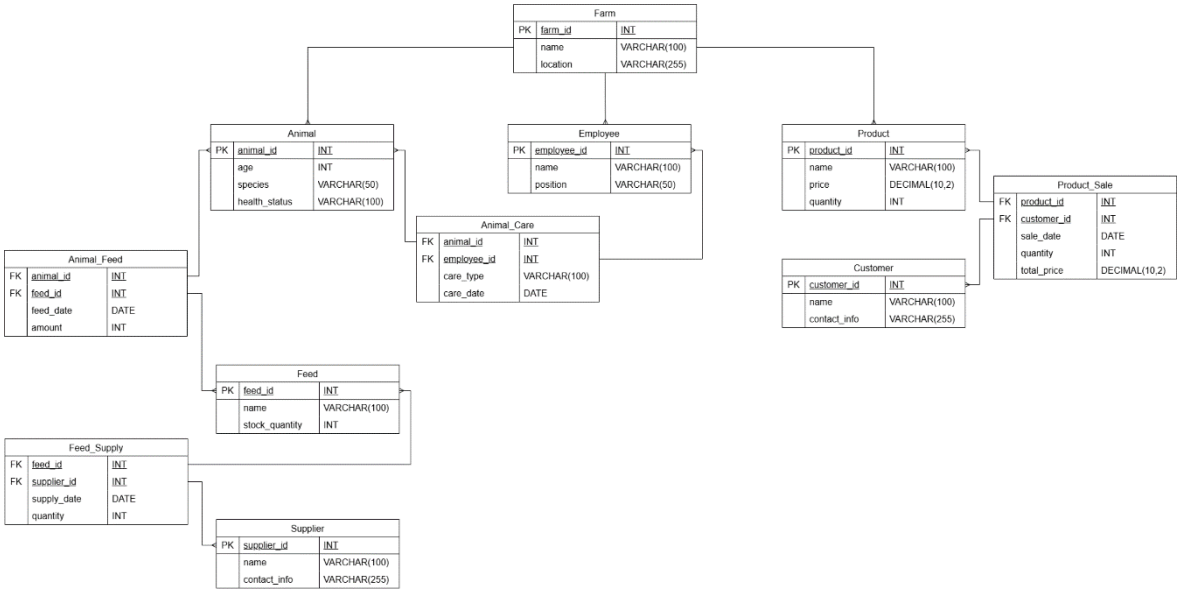
Vytvorte 1 ľubovoľný trigger okrem typu triggra uvedeného v predchádzajúcom bode (trigger musí obsahovať buď podmienku, cyklus alebo výnimku); 18

Záver 19

Úvod

V tomto projekte som sa venovala návrhu a implementácii relačnej databázy pre farmu. Cieľom zadania bolo vytvoriť databázový model, ktorý bude obsahovať tabuľky reprezentujúce entity ako napríklad zvieratá, zamestnancov, produkty a ďalšie súvisiace údaje. Súčasťou riešenia je implementácia SQL príkazov na vytváranie pohľadov, sekvencií, triggerov, ako aj testovanie funkcionality pomocou ukážkových dotazov.

Nižšie je zobrazená relačná schéma navrhnutej databázy:



Druhá časť zadania

1.1. 2 pohľady s netriviálnym selektom nad jednou tabuľkou (nestačí použiť iba vymenovanie stĺpcov, treba použiť "niečo navyše" napr.: vstavané funkcie);

```
-- POPIS: Tento pohľad zobrazuje priemerný vek zvierat pre každý druh.
-- Používa agregačnú funkciu AVG() a zaokrúhľuje výsledky na dve desatinné miesta.
CREATE VIEW average_animal_age_by_species AS
SELECT
    species,
    ROUND(AVG(age), 2) AS average_age
FROM
    Animal
GROUP BY
    species;
SELECT * FROM average_animal_age_by_species;
```

Tento SQL príkaz vytvára pohľad average_animal_age_by_species, ktorý zobrazuje priemerný vek zvierat pre každý druh (species) uložený v tabuľke Animal.

- Priemerný vek sa počíta pomocou agregačnej funkcie AVG(age).
- Výsledok sa zaokrúhľuje na 2 desatinné miesta pomocou funkcie ROUND().
- Údaje sú zoskupené podľa druhu (GROUP BY species), čím sa zabezpečí, že každý druh bude zastúpený jedným riadkom s vypočítaným priemerom veku.

SPECIES	AVERAGE_AGE
Cow	6
Duck	2
Pig	1.5
Goat	4
Sheep	3
Chicken	1
Horse	8

```sql
-- POPIS: Tento pohľad sumarizuje počet zvierat podľa ich zdravotného stavu.
-- Používa agregačnú funkciu COUNT() a GROUP BY pre zoskupenie podľa zdravotného stavu
CREATE VIEW animal_health_summary AS
SELECT
    health_status,
    COUNT(*) AS total_animals
FROM
    Animal
GROUP BY
    health_status;
SELECT * FROM animal_health_summary;
```

Tento SQL príkaz vytvára pohľad animal_health_summary, ktorý zobrazuje počet zvierat v databáze podľa ich zdravotného stavu (health_status).

- Funkcia COUNT(*) spočíta počet riadkov (zvierat) v každej skupine.
- GROUP BY health_status zabezpečuje zoskupenie podľa zdravotného stavu.
- Výsledný pohľad obsahuje dva stĺpce:
    - health_status – stav zdravia (napr. "zdravé", "choré", "v karanténe"),
    - total_animals – počet zvierat v danom stave.

HEALTH_STATUS	TOTAL_ANIMALS
Sick	2
Injured	2
Healthy	6

1.2. 3 pohľady so spájaním tabuliek (1x spojenie 3 tabuliek, 1x outer join, 1x spojenie aspoň 2 tabuliek);

```
-- POPIS: Tento pohľad zobrazuje podrobnosti o predajoch produktov,
-- vrátane mena zákazníka, názvu produktu, množstva a celkovej ceny.
-- Spája tabuľky Product, Product_Sale a Customer.
CREATE VIEW product_sales_details AS
SELECT
    c.name AS customer_name,
    p.name AS product_name,
    ps.quantity,
    ps.total_price,
    ps.sale_date
FROM
    Product_Sale ps
JOIN
    Product p ON ps.product_id = p.product_id
JOIN
    Customer c ON ps.customer_id = c.customer_id;
SELECT * FROM product_sales_details;
```

Tento pohľad product_sales_details zobrazuje detailné informácie o predajoch jednotlivých produktov vrátane zákazníkov, ktorí nákup uskutočnili. Slúži na prehľadné sledovanie predajných transakcií.

- Spája tri tabuľky:
    - Product_Sale – hlavná tabuľka predajov,
    - Product – informácie o predávaných produktoch,
    - Customer – informácie o zákazníkoch.
- Výstup obsahuje:
    - customer_name – meno zákazníka,
    - product_name – názov produktu,
    - quantity – množstvo kúpených kusov,
    - total_price – celková cena za daný predaj,

CUSTOMER_NAME	PRODUCT_NAME	QUANTITY	TOTAL_PRICE	SALE_DATE
Anna Malá	Milk	10	15	01-APR-24
Peter Veľký	Eggs	12	24	02-APR-24
Lucia Horváthová	Cheese	5	20	03-APR-24
Tomáš Bielik	Meat	7	38.5	04-APR-24
Eva Černá	Butter	6	19.2	05-APR-24
Roman Varga	Yogurt	8	14.4	06-APR-24
Martina Kováčová	Wool	4	10	07-APR-24
Igor Sýkora	Honey	3	18	08-APR-24
Barbora Farkašová	Chicken meat	9	40.5	09-APR-24
Jozef Hruška	Goat cheese	5	24	10-APR-24

```sql
-- POPIS: Tento pohľad zobrazuje kŕmenie zvierat vrátane názvu krmiva a množstva.
-- Spája tabuľky Animal_Feed a Feed.
CREATE VIEW animal_and_feed_usage AS
SELECT
    af.animal_id,
    f.name AS feed_name,
    af.feed_date,
    af.amount
FROM
    Animal_Feed af
JOIN
    Feed f ON af.feed_id = f.feed_id;
SELECT * FROM animal_and_feed_usage;
```

Pohľad animal_and_feed_usage slúži na zobrazenie údajov o kŕmení zvierat. Obsahuje informácie o tom, aké krmivo bolo použité, v akom množstve a v aký dátum, spolu s ID zvieraťa, ktoré bolo kŕmené.

- Spája dve tabuľky:

- Animal_Feed – uchováva informácie o dávkach krmiva pre jednotlivé zvieratá,
- Feed – obsahuje názvy dostupných krmív.

- Výstup obsahuje:
  - animal_id – identifikátor zvieraťa,
  - feed_name – názov použitého krmiva,
  - feed_date – dátum, kedy sa krmivo podalo,
  - amount – množstvo podaného krmiva.

ANIMAL_ID	FEED_NAME	FEED_DATE	AMOUNT
1	Hay	01-APR-24	5
2	Corn	01-APR-24	4
3	Grain	01-APR-24	3
4	Grass	01-APR-24	2
5	Barley	01-APR-24	6
6	Soy	01-APR-24	5
7	Wheat	01-APR-24	3
8	Mixed Feed	01-APR-24	4
9	Dry Grass	01-APR-24	2
10	Lucerne	01-APR-24	5

```sql
-- POPIS: Tento pohľad zobrazuje všetky zvieratá a informáciu o ich ošetrovaní (aj keď žiadne nemajú).
-- Používa LEFT JOIN medzi Animal a Animal_care.
CREATE VIEW animals_with_care_info AS
SELECT
    a.animal_id,
    a.species,
    a.health_status,
    ac.care_date,
    ac.care_type,
    ac.employee_id
FROM
    Animal a
LEFT JOIN
    Animal_care ac ON a.animal_id = ac.animal_id;
SELECT * FROM animals_with_care_info;
```

Tento pohľad animals_with_care_info zobrazuje všetky zvieratá vrátane informácií o ich ošetrovaní. Aj keď niektoré zvieratá ešte neboli ošetrované, budú v tomto pohľade zahrnuté vďaka použitiu LEFT JOIN.

- Spája dve tabuľky:
    - Animal – základná tabuľka so zvieratami,
    - Animal_care – obsahuje informácie o jednotlivých ošetreniach.
- Používa sa LEFT JOIN, aby sa zobrazili aj zvieratá, ktoré nemajú záznam o ošetrovaní.
- Výstup obsahuje:
    - animal_id – identifikátor zvieraťa,
    - species – druh zvieraťa,
    - health_status – zdravotný stav,
    - care_date – dátum ošetrenia (môže byť NULL),
    - care_type – typ vykonaného ošetrenia (môže byť NULL),
    - employee_id – zamestnanec, ktorý ošetrenie vykonal (môže byť NULL).

ANIMAL_ID	SPECIES	HEALTH_STATUS	CARE_DATE	CARE_TYPE	EMPLOYEE_ID
1	Cow	Healthy	01-APR-24	Vaccination	1
2	Pig	Sick	01-APR-24	Feeding	2
3	Sheep	Healthy	01-APR-24	Cleaning	3
4	Chicken	Healthy	01-APR-24	Health Check	4
5	Goat	Injured	01-APR-24	Feeding	5
6	Cow	Healthy	01-APR-24	Vaccination	6
7	Pig	Healthy	01-APR-24	Feeding	7
8	Horse	Sick	01-APR-24	Health Check	8
9	Duck	Healthy	01-APR-24	Feeding	9
10	Cow	Injured	01-APR-24	Treatment	10

1.3. 2 pohľady s použitím agregačných funkcií alebo zoskupenia;

```
-- POPIS: Tento pohľad zobrazuje počet ošetrení zvierat podľa zamestnanca.
-- Používa LEFT JOIN a COUNT(*).
CREATE VIEW employee_care_count AS
SELECT
    e.employee_id,
    e.name,
    COUNT(*) AS total_cares
FROM
    Employee e
LEFT JOIN
    Animal_care ac ON e.employee_id = ac.employee_id
GROUP BY
    e.employee_id, e.name;
SELECT * FROM employee_care_count;
```

Pohľad employee_care_count poskytuje štatistiku o počte ošetrení zvierat, ktoré vykonal každý zamestnanec. Je vhodný na sledovanie aktivity ošetrovateľov alebo veterinárov.

- Používa sa LEFT JOIN medzi tabuľkami:
    - Employee – obsahuje údaje o zamestnancoch,
    - Animal_care – obsahuje záznamy o jednotlivých ošetreniach zvierat.
- LEFT JOIN zabezpečuje, že aj zamestnanci, ktorí ešte nevykonali žiadne ošetrenie, budú zahrnutí (s počtom 0).
- COUNT(*) spočíta počet priradených záznamov v Animal_care.
- GROUP BY zabezpečí agregáciu podľa každého zamestnanca.

EMPLOYEE_ID	NAME	TOTAL_CARES
5	Tomáš Varga	1
8	Kristína Tóthová	1
6	Michaela Urbanová	1
7	Peter Hruška	1
2	Eva Nováková	1
9	Roman Farkaš	1
1	Ján Kováč	1
3	Marek Horváth	1
4	Lucia Bieliková	1
10	Zuzana Sýkorová	1

```sql
-- POPIS: Tento pohľad zobrazuje celkový počet predajov a tržby podľa mesiaca.
-- Používa TO_CHAR pre získanie roku a mesiaca.
CREATE VIEW monthly_sales_summary AS
SELECT
    TO_CHAR(sale_date, 'YYYY-MM') AS sale_month,
    COUNT(*) AS total_sales,
    SUM(total_price) AS total_revenue
FROM
    Product_Sale
GROUP BY
    TO_CHAR(sale_date, 'YYYY-MM');
SELECT * FROM monthly_sales_summary;
```

Pohľad monthly_sales_summary slúži na zobrazenie súhrnných informácií o predajoch za jednotlivé mesiace. Poskytuje prehľad o tom, koľko predajov sa uskutočnilo a aký bol ich celkový finančný objem (tržba).

- Zdrojom údajov je tabuľka Product_Sale, ktorá obsahuje jednotlivé záznamy o predajoch.
- Používa funkciu TO_CHAR(sale_date, 'YYYY-MM') na získanie formátu "rok-mesiac".
- Agregačné funkcie:
    - COUNT(*) – počet predajov v danom mesiaci,
    - SUM(total_price) – celková výška tržby v danom mesiaci.
- GROUP BY zabezpečuje zoskupenie podľa každého mesiaca.

SALE_MONTH	TOTAL_SALES	TOTAL_REVENUE
2024-04	10	223.6

1.4. 1 pohľad s použitím množinových operácií;

```
-- POPIS: Tento pohľad zobrazuje identifikátory zvierat, ktoré boli buď nakŕmené, alebo ošetrené, ale nie oboje naraz.
-- Používa množinové operácie UNION, INTERSECT a MINUS na porovnanie údajov z tabuliek ANIMAL_FEED a ANIMAL_CARE.
-- Výsledkom sú zvieratá, ktoré sa nachádzajú len v jednej z tabuliek – teda boli buď len kŕmené, alebo len ošetrované
CREATE VIEW Animals_Fed_Or_Cared_But_Not_Both AS
SELECT * FROM (
    (
        SELECT animal_id FROM ANIMAL_FEED
        UNION
        SELECT animal_id FROM ANIMAL_CARE
    )
    MINUS
    (
        SELECT animal_id FROM ANIMAL_FEED
        INTERSECT
        SELECT animal_id FROM ANIMAL_CARE
    )
);
SELECT * FROM Animals_Fed_Or_Cared_But_Not_Both;
```

Pohľad Animals_Fed_Or_Cared_But_Not_Both slúži na identifikáciu zvierat, ktoré boli buď **len kŕmené**, alebo **len ošetrované**, ale nie oboje naraz.

- Využíva množinové operácie:
    - UNION – spojí všetky jedinečné animal_id z oboch tabuliek,
    - INTERSECT – nájde zvieratá, ktoré sú v **oboch** tabulkách (teda boli aj kŕmené, aj ošetrované),
    - MINUS – z UNION výsledku odčíta výsledok INTERSECT, čím získa zvieratá, ktoré sú **len v jednej** z tabuliek.
- Výsledkom je zoznam animal_id, ktoré sa nachádzajú výhradne v jednej tabuľke.

no data found

Poznámka:

Ak databáza vráti „no data found“ pri vykonaní pohľadu, znamená to, že všetky zvieratá sú buď v oboch tabulkách, alebo v žiadnej. Inými slovami – neexistuje žiadne zviera, ktoré bolo len kŕmené alebo len ošetrované. Toto správanie je správne a očakávané v prípade, že neexistujú „exkluzívne“ záznamy.

1.5. 2 pohľady s použitím netriviálnych vnorených selektov;

```
-- POPIS: Tento pohľad zobrazuje zvieratá, ktoré boli kŕmené viac ako je priemerný počet kŕmení na zviera.
-- Používa vnorený selekt s agregáciou na výpočet priemeru a porovnanie s počtom kŕmení jednotlivých zvierat.
CREATE VIEW Animals_With_Above_Average_Feed AS
SELECT animal_id, COUNT(*) AS feed_count
FROM ANIMAL_FEED
GROUP BY animal_id
HAVING COUNT(*) > (
    SELECT AVG(feed_times) FROM (
        SELECT animal_id, COUNT(*) AS feed_times
        FROM ANIMAL_FEED
        GROUP BY animal_id
    )
);
SELECT * FROM Animals_With_Above_Average_Feed;
```

Pohľad Animals_With_Above_Average_Feed identifikuje zvieratá, ktoré boli **kŕmené častejšie** ako je **priemerný počet kŕmení na zviera**.

- Vnútorný SELECT (SELECT animal_id, COUNT(*) AS feed_times) spočíta počet kŕmení pre každé zviera.
- Potom sa z toho vypočíta **priemer kŕmení** cez AVG(feed_times).
- Vonkajší SELECT získava všetky animal_id, ktoré tento priemer **prekračujú**.

Tento pohľad je užitočný napríklad pri sledovaní špecifických prípadov (napr. zvieratá s nadmernou spotrebou krmiva).

no data found

Poznámka:

Ak databáza vráti „no data found“, znamená to, že žiadne zviera nebolo kŕmené častejšie než je priemer. Toto je korektné a zodpovedá realite vstupných dát.

```
-- POPIS: Tento pohľad zobrazuje zvieratá, ktoré nikdy neboli ošetrené zamestnancom
-- Používa vnorený selekt v klauzule WHERE s operátorom NOT IN.
CREATE VIEW Animals_Not_Cared_By_Employee_1 AS
SELECT animal_id
FROM ANIMAL
WHERE animal_id NOT IN (
    SELECT animal_id
    FROM ANIMAL_CARE
    WHERE employee_id = 1
);
SELECT * FROM Animals_Not_Cared_By_Employee_1;
```

Pohľad Animals_Not_Cared_By_Employee_1 zobrazuje všetky zvieratá z tabuľky ANIMAL, ktoré **neboli nikdy ošetrené** konkrétnym zamestnancom – v tomto prípade tým, ktorého employee_id = 1.

- Vnorený dotaz v WHERE získava všetky animal_id, ktoré **ošetroval** zamestnanec s ID 1.
- Vonkajší dotaz potom pomocou NOT IN zistí zvieratá, ktoré **nie sú v tomto zozname**, čiže neboli ním ošetrované vôbec.
- Týmto spôsobom vieš jednoducho identifikovať zvieratá, ktoré mal na starosti niekto iný alebo ešte neboli ošetrené vôbec.

ANIMAL_ID
2
3
4
5
6
7
8
9
10

Vytvorte 1 sekvenciu na generovanie primárnych kľúčov a trigger, ktorý bude vkladať hodnoty do príslušných tabuliek;

```
CREATE SEQUENCE seq_primary_keys
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

CREATE OR REPLACE TRIGGER trg_farm_id
BEFORE INSERT ON Farm
FOR EACH ROW
BEGIN
  IF :NEW.farm_id IS NULL THEN
    SELECT seq_primary_keys.NEXTVAL INTO :NEW.farm_id FROM dual;
  END IF;
END;
/
```

Táto časť kódu zabezpečuje, že pri vkladaní nového záznamu do tabuľky Farm:

- **Ak používateľ nezadá hodnotu farm_id**, systém ju automaticky vygeneruje pomocou sekvencie seq_primary_keys.
- Sekvencia zaručuje **jedinečnosť a automatické navyšovanie** hodnôt pre primárny kľúč.
- Trigger trg_farm_id sa spúšťa **pred každým INSERT-om** a kontroluje, či je :NEW.farm_id NULL. Ak áno, priradí hodnotu z seq_primary_keys.NEXTVAL.

```
Sequence created.

Trigger created.
```

Vytvorte 1 ľubovoľný trigger okrem typu triggra uvedeného v predchádzajúcom bode (trigger musí obsahovať buď podmienku, cyklus alebo výnimku);

```
CREATE OR REPLACE TRIGGER trg_check_animal_age
AFTER INSERT ON Animal
FOR EACH ROW
DECLARE
    ex_invalid_age EXCEPTION;
BEGIN
    IF :NEW.age < 0 THEN
        RAISE ex_invalid_age;
    END IF;
EXCEPTION
    WHEN ex_invalid_age THEN
        RAISE_APPLICATION_ERROR(-20001, 'Vek zvieraťa nemôže byť záporný.');
END;
/
```

Trigger trg_check_animal_age sa vykoná **po každom vložení** záznamu do tabuľky Animal a slúži na **validáciu dát**:

- Kontroluje, či novovložený záznam obsahuje **záporný vek (age < 0)**.
- Ak áno, vyvolá výnimku (ex_invalid_age) a použije systémový príkaz RAISE_APPLICATION_ERROR, ktorý:
    - Zastaví vykonanie SQL operácie,
    - Vypíše vlastnú chybovú hlášku: **„Vek zvieraťa nemôže byť záporný.”**
- Týmto sa zabezpečí, že v databáze nebudú uložené nekorektné údaje.

```
Trigger created.
```

Záver

Netriviálne selekty	2
Spájanie tabuliek	3
Použitie agregačných funkcií alebo zoskupenia	2
Množinová operácia	0
Vnorerené selekty	1
Sekvencia + trigger	2
Trigger	2
Celkovo	12